■ End-to-End ML Pipeline
Telco Customer Churn Prediction

Comprehensive Analysis Report

Source: telco_churn_ml_pipeline.ipynb · Analyzed by Claude

7,043	Customers	21	Features	2	Models	5-Fold	CV Strategy

1. Project Overview

This notebook builds a **production-ready, end-to-end machine learning pipeline** for predicting customer churn in a telecommunications company. Using the IBM Telco Customer Churn dataset, two classifiers — **Logistic Regression** and **Random Forest** — are trained inside Scikit-learn Pipelines with full preprocessing, hyperparameter tuning via GridSearchCV, thorough evaluation, and joblib-based model serialisation.

Primary Objective

Predict whether a customer will churn (cancel their subscription) based on demographic, account, and service-usage features.

Pipeline Stages at a Glance

- Data Loading & Exploration
- Data Preprocessing (Scaling + Encoding)
- Scikit-learn Pipeline Construction
- Hyperparameter Tuning with GridSearchCV
- Model Evaluation (metrics + visualisations)
- Pipeline Export with *joblib*
- Reload & Prediction Demo

2. Requirements

2.1 Python Version

Python 3.8 or later is recommended (tested with 3.10+).

2.2 Core Library Dependencies

Library	Purpose	Install Command
numpy	Numerical arrays & math	`pip install numpy`
pandas	DataFrame I/O, EDA, manipulation	`pip install pandas`
scikit-learn	Pipelines, models, metrics, CV	`pip install scikit-learn`
matplotlib	Plotting & chart export	`pip install matplotlib`
seaborn	Statistical visualisations	`pip install seaborn`
joblib	Model serialisation / deserialisation	`pip install joblib`

■ *All libraries are available via pip. No GPU required — all computation is CPU-based.*

2.2 Data Requirement

The pipeline expects the **Telco Customer Churn CSV** file. It can be supplied in two ways:

- **Local file** — place *WA_Fn-UseC_-Telco-Customer-Churn.csv* in the working directory.

- **Auto-download** — if the local file is absent, the notebook fetches it automatically from the IBM GitHub mirror.

2.3 Hardware / Environment

- Any modern laptop or desktop (CPU-only; no GPU needed).
- Minimum ~512 MB RAM; dataset is ~7 k rows and fits easily in memory.
- Jupyter Notebook, JupyterLab, or VS Code with the Jupyter extension.
- Internet access for auto-downloading the dataset (optional).

3. Dataset Details

3.1 About the Dataset

The **IBM Telco Customer Churn dataset** contains information about 7,043 customers of a fictional California-based telecom provider. Each row represents one customer; the target column is **Churn** (Yes / No).

Attribute	Value
Total rows	7,043
Total columns	21 (20 features + 1 target)
Churn rate	~26.5% (Yes) / ~73.5% (No)
Numerical features	3 — tenure, MonthlyCharges, TotalCharges
Categorical features	17 — gender, Partner, Dependents, PhoneService, …
Missing values	<1 row in TotalCharges (whitespace → NaN after coercion)
Identifier column	customerID (dropped before modelling)

3.2 Feature Groups

Features fall into four logical groups:

- **Demographics** — gender, SeniorCitizen, Partner, Dependents
- **Account info** — tenure, Contract, PaperlessBilling, PaymentMethod, MonthlyCharges, TotalCharges
- **Phone services** — PhoneService, MultipleLines
- **Internet services** — InternetService, OnlineSecurity, OnlineBackup, DeviceProtection, TechSupport, StreamingTV, StreamingMovies

4. Step-by-Step Pipeline Walkthrough

Step 1 — Import Libraries

All required packages are imported at the top of the notebook. Matplotlib display parameters (white background, grid, font size) are configured globally for consistent charts.

■ *Key imports: numpy, pandas, matplotlib, seaborn, joblib, and the full scikit-learn sub-modules listed in Section 2.*

Step 2 — Data Loading & Exploration

- Attempts to load from a **local CSV** first; falls back to downloading from GitHub.
- Prints dataset shape, column dtypes, and missing value counts.
- Visualises the **target distribution** (bar chart + pie chart) and saves *churn_distribution.png*.
- Class imbalance: ~73.5% No-Churn vs ~26.5% Churn — noted but not corrected in this baseline pipeline.

Step 3 — Data Preprocessing

Action	Detail
Fix TotalCharges dtype	pd.to_numeric(..., errors="coerce") — converts whitespace strings to NaN
Drop customerID	Identifier column; adds no predictive value
Encode target	"Yes" → 1, "No" → 0 using boolean cast
Impute NaN	Fill TotalCharges NaN with column median (<1 affected row)
Feature split	Separate X (features) and y (target); identify numerical vs categorical cols
Correlation heatmap	Plots & saves correlation_heatmap.png for numerical features + target
Train / Test split	80% train / 20% test, stratified on y, random_state=42

Step 4 — Building the Scikit-learn Pipeline

Two fully encapsulated pipelines are constructed — one per classifier. Each pipeline contains **two stages**: a ColumnTransformer preprocessor and the classifier itself.

Pipeline Component	Sub-step	Transformer / Estimator
Numerical sub-pipeline	Imputation	SimpleImputer(strategy="median")
	Scaling	StandardScaler()
Categorical sub-pipeline	Imputation	SimpleImputer(strategy="most_frequent")
	Encoding	OneHotEncoder(handle_unknown="ignore", sparse_output=False)
ColumnTransformer	Combines both	Applies each sub-pipeline to correct column set
Classifier A	—	LogisticRegression(max_iter=1000, random_state=42)
Classifier B	—	RandomForestClassifier(random_state=42, n_jobs=-1)

■ *Using Scikit-learn Pipelines prevents data leakage: the preprocessor is fitted only on training data and applied to test data via transform(), never fit_transform().*

5. Hyperparameter Tuning

Both pipelines are optimised using **GridSearchCV** with **StratifiedKFold (5 splits)**. The scoring metric is **ROC-AUC**, which is suitable for imbalanced binary classification. All CPU cores are used (n_jobs=-1) for parallelism.

5.1 Logistic Regression — Hyperparameter Grid

Parameter	Values Searched	Meaning
C	`[0.01, 0.1, 1, 10]`	Inverse regularisation strength (smaller = more regularisation)
penalty	`['l1', 'l2']`	Regularisation type — L1 (Lasso) or L2 (Ridge)
solver	`['liblinear', 'saga']`	Optimisation algorithm (both support L1 & L2)

■ *Total LR combinations: 4 × 2 × 2 = 16 parameter sets × 5 folds = 80 fits.*

5.2 Random Forest — Hyperparameter Grid

Parameter	Values Searched	Meaning
n_estimators	`[100, 200]`	Number of decision trees in the ensemble
max_depth	`[None, 10, 20]`	Maximum depth of each tree (None = unlimited)
min_samples_split	`[2, 5]`	Min samples needed to split an internal node
max_features	`['sqrt', 'log2']`	Features to consider at each split

■ *Total RF combinations: 2 × 3 × 2 × 2 = 24 parameter sets × 5 folds = 120 fits.*

6. Model Evaluation

6.1 Metrics Computed

- **Accuracy** — overall fraction of correct predictions
- **Precision** — of predicted churners, how many actually churned
- **Recall** — of actual churners, how many were caught
- **F1 Score** — harmonic mean of precision and recall
- **ROC-AUC** — area under the Receiver Operating Characteristic curve (primary metric)

6.2 Visualisations Generated

File Saved	Chart Type	What It Shows
`churn_distribution.png`	Bar + Pie	Class balance of the target variable
`correlation_heatmap.png`	Heatmap	Pearson correlations among numerical features & Churn
`model_evaluation.png`	2×2 Grid	Confusion matrices + ROC curves for both models
`metrics_comparison.png`	Grouped Bar	Side-by-side comparison of all 5 metrics

`feature_importances.png`	Horizontal Bar	Top 20 Random Forest feature importances (Gini)

6.3 Feature Importance Analysis

After tuning, the Random Forest's feature importances are extracted. Because the pipeline applies OneHotEncoding to categorical features, the notebook reconstructs full feature names using *get_feature_names_out()* on the encoder. The top 20 most important features are plotted (Gini impurity-based importance).

■ *Key features typically driving churn: tenure, MonthlyCharges, TotalCharges, Contract type, and InternetService type.*

7. Model Export & Reload

7.1 Saving Pipelines with joblib

After evaluation, **three pipeline files** are saved to the *saved_models/* directory:

- **logistic_regression_pipeline.joblib** — best LR estimator from GridSearchCV
- **random_forest_pipeline.joblib** — best RF estimator from GridSearchCV
- **best_pipeline.joblib** — whichever model achieved the higher ROC-AUC

■ *The entire Pipeline object is serialised — including the ColumnTransformer preprocessor fitted on training data. No separate scaler or encoder file is needed.*

7.2 Reload & Prediction Demo

The notebook demonstrates a full reload-and-predict cycle to confirm the saved pipeline is production-ready:

- Load *best_pipeline.joblib* using **joblib.load()**.
- Sample 5 rows from the test set.
- Call **.predict()** and **.predict_proba()** on raw (unscaled) input.
- Display True_Churn, Predicted_Churn, and Churn_Prob side by side.

8. Important Considerations & Best Practices

Data Leakage Prevention	All preprocessing (scaling, encoding, imputation) is performed inside the Pipeline, fitted on training data only. The test set is never seen during fitting.
Stratified Splitting	Both train_test_split and StratifiedKFold use stratify=y to maintain the original class distribution in every split — critical with 26.5% minority class.
Class Imbalance	The dataset has a ~74% / 26% split. No re-sampling (SMOTE, class_weight) is applied in this baseline. For production, consider class_weight="balanced" or oversampling.
Scoring Metric	ROC-AUC is used for tuning rather than accuracy, which is appropriate for imbalanced classes where a naive "predict always No-Churn" achieves 73% accuracy.
Feature Importance Caveat	Gini importance can overestimate high-cardinality or continuous features. Permutation importance or SHAP values provide more reliable attributions.
Reproducibility	random_state=42 is set on train_test_split, StratifiedKFold, LogisticRegression, and RandomForestClassifier — ensuring bit-identical results on re-runs.
Sparse Output Disabled	OneHotEncoder is initialised with sparse_output=False, returning a dense NumPy array. This is compatible with all downstream Scikit-learn estimators.
Grid Search Compute Time	LR grid: 80 fits. RF grid: 120 fits. RF is significantly slower; on a typical laptop expect 2–5 minutes for the RF grid with n_jobs=-1.

Model Selection Logic	The "best" model is selected programmatically by comparing ROC-AUC scores — not hardcoded — so the script remains valid if a future model outperforms RF.
Production Readiness	The exported joblib pipeline accepts raw pandas DataFrames with the original column names. No manual preprocessing is needed at inference time.

9. Outputs & Artefacts

Artefact	Type	Location / Notes
`churn_distribution.png`	Image	Working directory — class balance charts
`correlation_heatmap.png`	Image	Working directory — numerical feature correlations
`model_evaluation.png`	Image	Working directory — CM + ROC for both models
`metrics_comparison.png`	Image	Working directory — grouped bar of all 5 metrics
`feature_importances.png`	Image	Working directory — top-20 RF importances
`saved_models/logistic_regression_pipeline.joblib`	Model	Serialised LR pipeline
`saved_models/random_forest_pipeline.joblib`	Model	Serialised RF pipeline
`saved_models/best_pipeline.joblib`	Model	Best model by ROC-AUC

10. Suggested Improvements

■ **Handle class imbalance** — Add class_weight="balanced" to classifiers or apply SMOTE oversampling.

■ **Expand model zoo** — Add XGBoost, LightGBM, or a stacking ensemble for potentially higher AUC.

■ **Use SHAP for explainability** — Replace Gini importance with SHAP values for more trustworthy feature attribution.

■ **Add cross-validated metrics** — Report mean ± std of all 5 metrics across folds, not just the test set.

■ **Threshold optimisation** — Tune the classification threshold (default 0.5) to balance precision / recall for the business use case.

■ **MLflow / Weights & Biases** — Log experiments, parameters, and metrics for reproducible tracking.

■ **CI/CD pipeline** — Wrap the notebook as a Python script and add automated tests and a Docker container for deployment.